Hand Retargeting for the MABEL ORCA Hands:
Keypoint IK (Gauss–Newton and SLSQP), Vector Retargeting, Anatomical Mapping, and the Grasp Map (`mabel.hand_retargeting`)

MABEL Project — Control & Teleoperation

July 22, 2026

Abstract

Hand control on MABEL is *position-controlled and fully decoupled* from the rest of the body: whatever the arms, torso, or base are doing (kinematic IK, impedance, compliance), the fingers always track through this subpackage. The input is a device-agnostic hand skeleton — Vision Pro ARKit joints or a webcam hand-landmark skeleton, normalised by `mabel.sources` — so the retargeters never know which device produced it. We document the canonical palm-frame construction that makes the mapping scale- and chirality-free, the DexPilot-style keypoint IK cost and its two solvers (a damped Gauss–Newton with analytic Jacobian at ∼0.5 ms, and the reference SLSQP cosine objective), the AnyTeleop-style vector variant with the per-user hand-size α, the direct anatomical mapping, and the one-scalar grasp map. Benchmarks from the 2026-07 weight sweep: cold-start curl similarity 0.89, fist 0.87, pinch error 2.6 mm at 8.8 ms per solve.

Keywords: dexterous teleoperation, hand retargeting, fingertip IK, DexPilot, AnyTeleop, ORCA hand.

1 Design position

- **Decoupled by contract.** Finger joints never appear in any whole-body law; conversely the hand solvers write only finger actuators (verified in `tests/test_retargeting.py`). The ORCA *wrist* is part of the arm chain (`whole_body_retargeting.pdf`), not of this package.
- **Source-agnostic.** Both ARKit's named 27-joint skeleton and the 21-landmark webcam convention land on the same named-matrix dict before any solver runs.
- **Position servos downstream.** Solved targets stream to per-joint position actuators ($\tau = k_p(q^\star - q) - k_d\dot{q}$); the smoothness term + warm start regularise, the servo tracks. The same $q^\star$ feed the real hand's PD loop unchanged.

Four interchangeable retargeters; `make_hand_retargeter(model, data)` builds the configured one (`config.FINGER_RETARGET`, solver `config.HAND_SOLVER`).

2 Canonical palm frame and normalization

Both skeletons (human landmarks and robot FK) are expressed in a palm frame built *identically* on each side: origin at the wrist, $\hat{y}$ toward the middle knuckle, palm normal $\hat{z} \propto$ (index-knuckle ray) $\times$ (pinky-knuckle ray), $\hat{x}$ completing the right-handed triad; all points are then

divided by the palm length. Identical construction makes the mapping scale-free and chirality-free. Additionally each *human* finger is length-scaled about its knuckle so its normalised bone length matches the robot finger's — a straight human finger maps to a straight ORCA finger rather than a curled one. With a calibrated `OperatorProfile`, measured per-finger lengths replace the live estimate (`calibration.pdf`).

3 Keypoint IK (`keypoint_ik.py`, default)

Per hand, the 16 finger joints q minimise

$$\begin{aligned} E(q) \;=\; & W_{\text{shape}} \sum_i \left\| p_i^{\text{rob}}(q) - p_i^{\text{hum}} \right\|^2 \;+\; W_{\text{dir}} \sum_b \left\| \hat{u}_b^{\text{rob}}(q) - \hat{u}_b^{\text{hum}} \right\|^2 \\ & + \sum_f W_{\text{pinch}}(f) \left\| \left(p_{\text{thumb}} - p_f\right)^{\text{rob}} - \left(p_{\text{thumb}} - p_f\right)^{\text{hum}} \right\|^2 \\ & + W_{\text{smooth}} \|q - q_{\text{prev}}\|^2 \;+\; W_{\text{reg}} \|q\|^2, \end{aligned} \tag{1}$$

subject to joint bounds (projected). The terms, and why each exists:

- *Shape* (W=15): normalised keypoint positions (MCP, PIP, and an extrapolated tip at 0.85× the last phalanx) — the workhorse.
- *Bone direction* (W=30): per-bone unit vectors (knuckle→PIP, PIP→tip). Human and ORCA position manifolds cannot coincide everywhere; the direction rows drive flexion even there. Measured to rescue deep curls and cold starts the position rows leave jammed at a bound corner; the 2026-07 sweep found W_{dir}=30/W_{shape}=15 dominant on every metric.
- *Gated pinch* (the DexPilot trick [1]): the thumb–fingertip *relative* vector is weighted W=50 only while the human pinch distance for that pair is small (normalised < 0.55), and W=0 otherwise — an always-on pull toward the thumb measurably costs ∼0.07 of fist flexion. When ≥ 3 pairs "pinch" simultaneously the gesture is a fist, not a pinch: the pinch rows are dropped and shape + direction close the grasp.
- *Smoothness / regularisation*: warm-started temporal continuity.

Gauss–Newton solver. Each hinge j with world axis $\hat{\omega}_j$ and centre c_j contributes the analytic point Jacobian column $\partial p_i / \partial q_j = \hat{\omega}_j \times (p_i - c_j)$ (for j an ancestor of p_i). Stacking residuals $r(q)$ with row weights $\sqrt{W}$, each damped Gauss–Newton iteration solves the normal equations

$$\left(J^\top J + \mu I\right) \Delta q = -\, J^\top r, \qquad \mu = 10^{-2}, \tag{2}$$

4 iterations, bounds projected each step; one iteration costs one analytic mini-FK of the 17-body hand subtree plus a 16×16 solve (∼0.5 ms total, vs ∼40 ms for SLSQP's numerical Jacobian, which starved the physics loop's GIL). A corner-jam detector restarts from rest when the previous solution is pinned at bounds with no descent.

SLSQP reference objective. `solver="slsqp"` minimises the dexterous-hand reference cost instead — cosine similarity of the stacked keypoints plus the relative-pinch, smoothness and regularisation terms (W = 20/150/0.5/0.02, 5 iterations, warm-started) — kept for parity with the apps' `avp_teleop` planner.

Threading. Solving runs on a background worker per retargeter (both hands) paced at 20 Hz; `apply()` deposits the newest skeleton and streams the latest solved targets, so the physics loop never blocks (`mj_step` releases the GIL; solver and physics overlap). Tests use `synchronous=True` for determinism.

4 Vector retargeting (`vector.py`)

The AnyTeleop-style [2] variant replaces the position rows of Eq. (1) with *scale-invariant keypoint vectors*: robot vectors are matched to α-scaled human vectors, where the per-user α (robot hand size / operator hand size) comes from the calibrated profile. Implemented as a subclass of the keypoint retargeter (same GN machinery, different residual assembly); benchmarked head-to-head in `exp_hand_retarget.py`.

5 Anatomical mapping (`anatomical.py`)

The v1 mapper: extract per-joint flexion/abduction angles from the skeleton geometry (angles between bone vectors, signed in the palm plane) and write them to the corresponding actuators through a fixed linear map + smoothing. Cheapest and dependency-free, but it did not transfer well to the real hand (no manifold correction, no pinch objective) — superseded by §3, kept for comparison and as a fallback.

6 Grasp map (`grasp.py`)

The app slider / pinch-to-grasp path: one scalar $g \in [0, 1]$ curls all fingers linearly from open to fist inside each actuator's ctrlrange (verified monotone in the tests). The operator-side pinch distance maps to g through the calibrated pinch span (open 8 cm $\rightarrow$ g=0, closed 2 cm $\rightarrow$ g=1, per-user values from the profile).

7 Experiments

metric	source	result
cold-start curl similarity	2026-07 weight sweep	0.89
fist similarity	sweep	0.87
pinch error	sweep	2.6 mm
solve time (GN, per hand)	sweep	8.8 ms
curl transfer sim$\rightarrow$real trend	`exp_hand_retarget.py`	Fig. 1
targets in ctrlrange, body untouched	pytest	pass
grasp monotone in g	pytest	pass

7.1 The live engine's finger path (2026-07-21)

The teleop engine (`scripts/teleop/engine.py`, what the apps drive through the facade) now runs *this* keypoint IK as its default finger solver — the dexterous-hand planner lineage (`planner_avp.py`/`planner_per_finger.py` $\rightarrow$ `keypoint_ik.py`) — via the synchronous adapter `scripts/teleop/hand_keypoint.py`, replacing the webcam-parity anatomical decomposition (kept

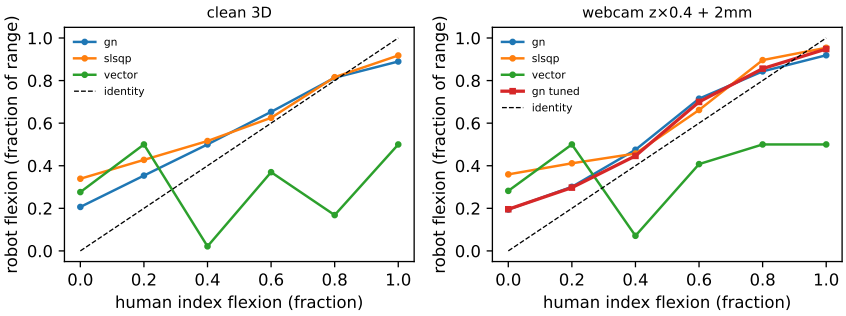


Figure 1: Curl-transfer benchmark across retargeters (`experiments/exp_hand_retarget.py`); per-finger calibration effects in `figures/calib_fingers.pdf` (see `calibration.pdf`).

as fallback, `MABEL_FINGER_SOLVER=anatomical`). Solves are paced at 30 Hz (the reference planner's design; servos track in between) and one-euro filtered. Engine-level solver battery (`tools/exp_retarget_fidelity.py`; MuJoCo-FK metrics with the 22 mm pad-extended fingertips, so absolute gaps carry a tip-definition offset — the cross-solver comparison is the claim):

metric	keypoint GN	keypoint SLSQP	anatomical
fist flexion fraction at full curl	0.87	0.80	0.99
pinch gap, index/pinky [mm]	19/26	36/62	25/76
left–right mirror inconsistency [rad]	0.000	0.160	0.098
output noise σ at 2 mm input noise [rad]	0.057	0.068	0.081
solve p50/p95 [ms]	3.5/4.3	46/53	0.8/0.9

The anatomical mapper's higher raw curl is a direct angle copy; it pays for it with 2–3× worse ring/pinky pinch closure, chiral asymmetry, and noisier output — the pinch geometry is what grasping cares about. Degenerate-palm, sparse-skeleton and collapsed-input frames hold the last pose without faulting for all three solvers. Since 2026-07-22 the keypoint GN runs 2 Gauss–Newton iterations when warm-started and 4 on cold starts (`MABEL_HAND_GN_ITERS_WARM`), the corner-escape re-solve is gated to cold starts only, and the two hands' 30 Hz solve cadences are phase-staggered by half a period so they never spike the same engine frame. Per-hand solve fell from 6.3 to 3.5 ms p50 (4.3 ms p95) with every quality row above re-measured unchanged.

8 Implementation & usage

```
from mabel.hand_retargeting import make_hand_retargeter, apply_grasp
rt = make_hand_retargeter(model, data)  # config-selected variant
rt.apply("Left", pose.left_joints)       # named skeleton -> servos
apply_grasp(model, data, "Right", 0.7)   # one-scalar grip
```

Legacy imports (`mabel.hand_ik_retarget`, `mabel.hand_vector_retarget`, `mabel.hand_retarget`, `mabel.hands`) resolve here through shims.

9 Limitations

- Human and ORCA fingertip manifolds cannot coincide; the direction rows mitigate but residual shape error is irreducible (curl similarity < 1).

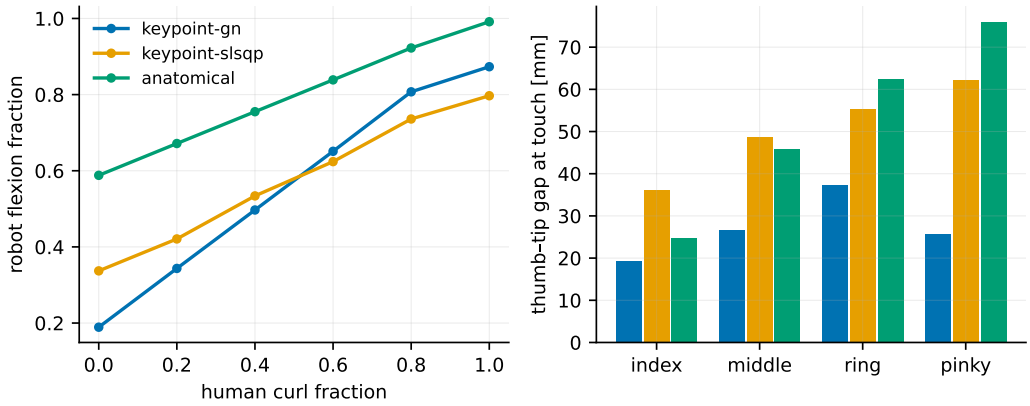


Figure 2: Engine-level finger-solver comparison: curl transfer (left) and thumb–fingertip gap at a true human touch (right).

- Webcam skeletons are noisier than ARKit's; the smoothness term is tuned for ARKit rates.
- The asynchronous worker introduces up to one solve period (50 ms) of finger latency; acceptable for grasping, not for dynamic in-hand manipulation.
- The anatomical mapper is kept only as a baseline; do not deploy it on hardware.

References

[1] A. Handa et al., "DexPilot: Vision-based teleoperation of dexterous robotic hand-arm system," *Proc. IEEE ICRA*, 2020.

[2] Y. Qin et al., "AnyTeleop: A general vision-based dexterous robot arm-hand teleoperation system," *RSS*, 2023 (arXiv:2307.04577).